

Real-Time Voting System

CIS 266 | Instructor: Huda Judeh

Team Project Presentation

Project Goals

- Display a question with multiple answer choices
- Allow users to vote by clicking a button
- Update vote results live for all connected users - no page refresh
- Show results as animated progress bars
- Include a countdown timer that closes voting after 30 seconds
- Include a Reset Poll button that resets votes for all users
- Use Socket.IO for all real-time communication between client and server

Methods & Tech Stack

Server Side

- Node.js — JavaScript runtime environment
- Express — serves the HTML file to the browser
- Socket.IO (server) — handles connections, listens for vote events, broadcasts results

Client Side

- HTML / CSS / JavaScript — no framework needed
- Socket.IO (client) — connects to server, emits votes, receives result updates
- Progress bars — updated dynamically via JavaScript on every result event

How It Works

1. User opens the app in their browser. Socket.IO automatically establishes a WebSocket connection to the server.
2. User clicks a vote button. The client emits a `vote` event to the server with the chosen option.
3. Server receives the `vote` event, increments the count for that option in memory.
4. Server emits an `updateResults` event to ALL connected clients, including the sender.
5. Every browser receives the updated vote counts and re-draws the progress bars instantly.

Code Illustration

server.js — Socket.IO event handling

```
io.on('connection', (socket) => {  
  socket.emit('updateResults', votes);  
  
  socket.on('vote', (choice) => {  
    votes[choice]++;  
    io.emit('updateResults', votes);  
  });  
  
  socket.on('resetPoll', () => {  
    for (let k in votes) votes[k] = 0;  
    io.emit('updateResults', votes);  
  });  
});
```

- `io.emit()` — sends the event to every connected client
- `socket.emit()` — sends only to the client that triggered it
- `socket.on()` — listens for an incoming event by name

index.html — Client-side Socket.IO

```
const socket = io();  
  
function castVote(choice) {  
  if (hasVoted) return;  
  socket.emit('vote', choice);  
  hasVoted = true;  
}  
  
socket.on('updateResults', (votes) => {  
  updateBars(votes);  
});
```

Testing & Debugging

Functionality Tests

- Multi-tab test: open 2+ tabs at localhost:3000 - voting in one tab should update all others instantly
- Timer test: wait 30 seconds - buttons must disable and 'Voting closed' message should appear
- Reset test: click Reset Poll - all bars return to 0%, timer restarts, buttons re-enable
- Duplicate vote test: vote once - all buttons become disabled, user cannot vote again

Debugging

- Browser DevTools (F12) Console: check for socket connection messages and errors
- Server terminal: logs 'User connected' and 'User disconnected' for each client
- Network tab in DevTools: look for the WebSocket upgrade under the WS filter

Team Member Roles

Joshua Williams

Backend / Server

- Set up Node.js and Express server
- Configured Socket.IO on the server
- Implemented vote counting logic
- Handled broadcasting results to all clients
- Built the reset poll server-side logic

Jenn Woldt

Frontend & Docs

- Built the HTML layout and CSS styling
- Created vote buttons and live progress bars
- Connected client-side Socket.IO events
- Implemented countdown timer and UI feedback
- Wrote the README and prepared the presentation

Summary

What We Built

- A real-time voting web app using Node.js, Express, and Socket.IO
- Votes update instantly for all connected users without any page refresh
- Features include live progress bars and 30-second countdown timer, and a reset button

What We Learned

- How to use Socket.IO to emit and listen for events on both the client and server
- How to broadcast data from one client to all connected clients through the server
- How to update the DOM dynamically in response to real-time server events
- How to structure a full-stack Node.js project and divide work as a team